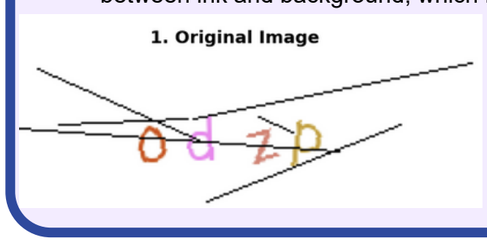
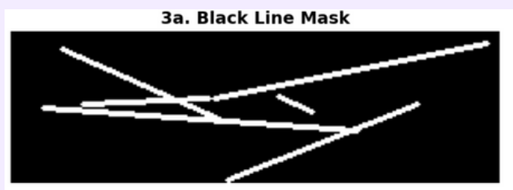
GROUP 29 CAPTCHA RECOGNITION

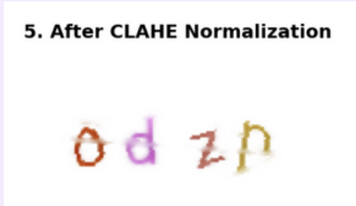
1.1 PREPROCESSING

DARRYL SHUCK/ KAI KIAT/ KENG HIAN

- Resize
  - We first resize each captcha to a fixed height, in our case, 80 pixels, while keeping aspect ratio. This makes stroke thickness stable and keeps area-based thresholds from varying too much across images.
- Edge-preserving Denoise
  - use a bilateral filter removes speckle noise but keeps edges sharp, preventing tiny noise from becoming false components later.
- Thin Black-line Removal
  - We then remove thin scanning or printing artifacts. We threshold the V channel in HSV to detect very dark lines, optionally dilate them, and then inpaint those regions. This prevents characters from being split by random thin lines.
- Contrast Normalization with CLAHE
  - We convert the image to LAB and apply CLAHE on the L channel. This fixes uneven lighting and increases the contrast between ink and background, which makes later clustering much more reliable.







1.2 K-MEANS CLUSTERING SEGMENTATION

DARRYL SHUCK

1. Convert Image to HSV
   - Separates color (Hue) from intensity (Value) for more stable analysis. This produces a colour representation that makes ink regions easier to distinguish.
2. Apply K-Means Clustering
   - Treat each pixel as a data point in HSV space. Cluster pixels into K colour groups based on similarity. Each cluster reflects a dominant colour region within the image.
3. Identify Likely Ink Clusters
   - Score each cluster using average Saturation (S) and Value (V) where score = $S \cdot (1-0.5V)$. Ink strokes usually appear in clusters with higher saturation or lower brightness. Select the top cluster(s) and generate a binary mask containing only ink pixels.
4. Extract and Filter Connected Components
   - Run connected components to group neighbouring ink pixels into segments. Obtain stats like bounding boxes, pixel counts, and shape size.
   - Filter components using minimum and maximum area thresholds to remove noise. Remaining components represent the final tokens.

1.3 PROJECTION SEGMENTATION

KENG HIAN

Uses vertical projection histograms to identify character boundaries.

Step 1: Converts images to grayscale

Step 2: Applies binary thresholding

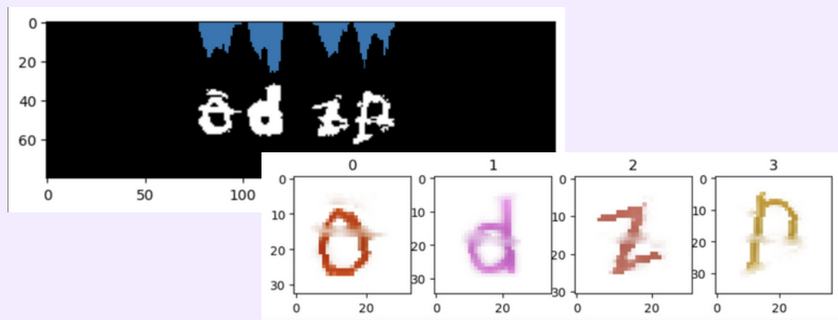
Step 3: Optionally Gaussian blur, destroy characters with weak signal

Step 4: Calculates column-wise pixel intensity projections to detect character regions where projection values exceed a threshold.

Step 5: Segment the original image into multiple images, removes vertical whitespace

Step 6: Splits wide segments again with higher threshold

Step 7: Padding tokens into square images and resize to (64, 64)



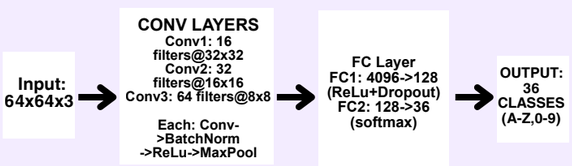
Performance: ~85% (based on number of token)

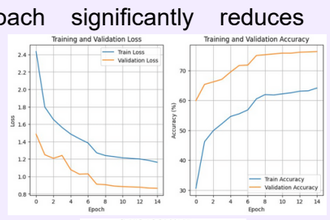
Shortcoming: Overlapping characters, wide characters with weak signal

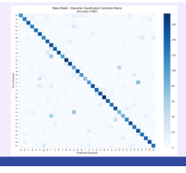
1.4 CNN MODEL

DARRYL/KAI KIAT

Key Innovation: Both segmentation methods run independently on each CAPTCHA. The system compares token counts from both methods, only when both agree does the pipeline proceed to character recognition, ensuring high-confidence segmentation. The CNN model (3-layer convolutional architecture with batch normalization and dropout) then classifies each 64×64 token into 36 classes (a-z, 0-9). This dual-validation approach significantly reduces segmentation errors before classification.







TOKEN MODEL CHARACTER ACCURACY: 0.6832 (68.32%)

2.1 IMAGE PREPROCESSING

XIANG RUI

Each CAPTCHA image is cleaned by removing thin lines with HSV-based inpainting, then converted to grayscale and resized to 150×50 pixels for consistency.

2.2 CRNN MODEL ARCHITECTURE

XIANG RUI/ NICHOLAS LAM

1. CNN feature extraction

Five conv layers (32 → 64 → 128 → 128 → 256 filters) extract hierarchical features, with later (2×1) pooling to reduce height while preserving width.

2. Sequence-ready feature map

Encoder outputs a 1×37×256 tensor where each of the 37 vertical slices is a 256-D time-step descriptor.

3. Sequence formation

The feature map is reshaped into a sequence of 37 time steps, each a 256-dimensional feature vector for the recurrent model.

4. Bidirectional LSTM encoding

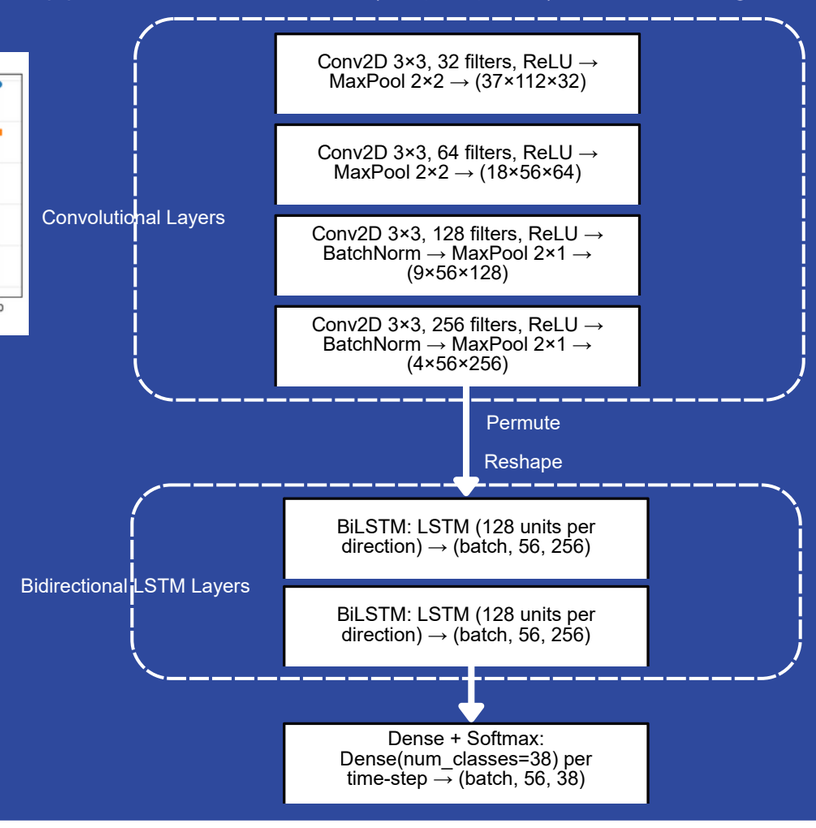
Two stacked BiLSTM layers (128 units per direction) encode context from both left and right to handle overlaps and variable spacing.

5. Character probability output

A time-distributed Dense + softmax produces per-time-step probabilities over 37 classes (A–Z, 0–9, blank) for CTC decoding.

TRAINING CHARACTER ACCURACY: 98.01%
VALIDATION CHARACTER ACCURACY: 74.81%

Early epochs tend to have fluctuating validation accuracy due to model parameters not being the most optimal. However, we implemented ReduceLROnPlateau as a training callback, which reduces learning rate with parameters factor=0.5 and patience=3.



2.3 CTC LOSS TRAINING & INFERENCE

XIANG RUI/ NICHOLAS LAM

Training uses Connectionist Temporal Classification (CTC) loss, which removes the need for character-level alignment by learning from all valid ways the model's time-step outputs can map to the target text. CTC naturally handles repeated characters and blank tokens, allowing paths like "AAA--BB-CC" to decode to "ABC". During inference, the model first performs greedy decoding, selecting the highest-probability character at each time step. The raw output is then passed through the CTC rules shown in the diagram: consecutive duplicates are collapsed, and blank tokens are removed, producing the final text sequence. This end-to-end approach allows the model to recognise variable-length CAPTCHAs without segmentation, with training stabilised using callbacks such as ModelCheckpoint and ReduceLROnPlateau.

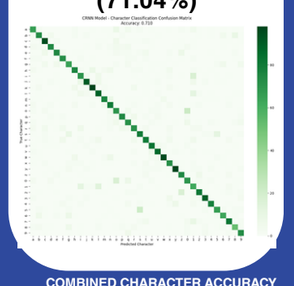
STEP 1 GREEDY DECODING
[37, 37, 6, 6, 6, 37, 11, 11, 37, 3, 3, 3, 37, ...]
[----,----, F, F, F, ----, K, K, ----, C, C,C, ----, ...]

STEP 2: COLLAPSE DUPLICATES
[----,----, F, F, F, ----, K, K, ----, C, C,C, ----, ...]
[-][F][-][K][-][C][-][D][-][6][-][C]...

STEP 3: REMOVE BLANKS
[-][F][-][K][-][C][-][D][-][6][-][C]...
[F][K][C][D][6][C]

FINAL OUTPUT: "FKCD6C"

CRNN CHARACTER ACCURACY: 0.7104 (71.04%)



COMBINED CHARACTER ACCURACY (TOKEN AND CRNN MODELS): 69.30%